

Deep Reinforcement Learning: Bipedal Walker with DQN

Introduction into Deep Learning - Project 21

Lea Seifried (12428595), Michael Hagmann (12426877), Dominik Rzecki (12410100)

A bipedal walker is a system designed to move using two legs. In robotics and AI, it is often framed as a classical control or machine learning control problem. A DQN architecture capable of enabling the Gymnasium (formerly OpenAI Gym) bipedal agent to locomote across its environment has been devised, trained and evaluated.

Contents

1	Introduction	1
1.1	Agent	1
1.2	Environment	2
1.3	Goal	2
2	Theory	2
2.1	Deep Neural Networks	2
2.1.1	Universal Function Approximators	2
2.1.2	Hierarchical Feature Representation	2
2.1.3	Scaling and Performance	2
2.2	Reinforcement Learning	2
2.2.1	Markov Decision Process - MDP	2
2.2.2	Return G_t	2
2.2.3	Policy $\pi(a_i s_i)$	3
2.2.4	World Models	3
2.2.5	Value Functions	3
2.2.6	Bellman Equation	3
2.2.7	Learning Methods	3
2.3	Deep Q-Networks (DQN)	3
2.3.1	DQN Objective and Training	4
2.3.2	Action Selection and Exploration	4
2.3.3	Experience Replay Buffer	4
2.3.4	Target Networks and the Moving Target Problem	4
3	Implementation	5
3.1	Configuration	5
3.2	Policy	5
3.3	Replay Memory	5
3.4	Learner	5
3.5	Neural Networks	5
3.6	Training Loop	5
3.7	Action Space Discretisation	6
4	Training	6
5	Results	6

1. Introduction

The Bipedal Walker environment is a classical reinforcement learning benchmark in which a simulated two-legged robot must learn to walk forward on a 2D terrain using continuous control. The system is part of the Box2D physics suite in Gymnasium (formerly OpenAI Gym) and can be used to evaluate locomotion policies under Newtonian mechanics, partial observability, and contact-rich interactions.

1.1. Agent

The agent is a four-joint bipedal robot consisting of a single rigid torso (“hull”) and two legs, each with two actuated joints (hip and knee). The policy controls joint motors that (in general) apply continuous motor speeds in the range $[-1, 1]$ for each of the four actuators (joints). Though specific control architectures like DQN, as we will see later, are only capable of applying a discrete subset of aforementioned motor ranges.

The agent’s observation vector is 24-dimensional and includes:

- Hull angle, angular velocity, and linear velocities (x and y directions)
- Joint angles and angular velocities for both legs
- Binary contact sensors indicating whether legs touch the ground
- 10 lidar range measurements providing local terrain awareness

The non-discretized action space is:

$$\mathcal{A} = [-1, 1]^4$$

The observation space is:

$$\mathcal{O} \in \mathbb{R}^{24}$$

The agent does not receive explicit global position coordinates, making the task partially observable.

1.2. Environment

The topology in its simplest form is a plain, obstacle-free, horizontal ground slab in 2D space, subject to Newtonian mechanics and gravity. The physics are simulated using Box2D and include, momenta, gravity, frictional contact dynamics between feet and terrain.

The environment provides two variants:

- **Normal:** slightly uneven terrain
- **Hardcore:** includes stumps, pits, and ladders requiring more advanced locomotion strategies

Each episode typically runs up to 1600 steps (normal) or 2000 steps (hardcore).

1.3. Goal

The goal is to make the agent walk forward along the terrain as far as possible without falling.

More formally, the objective is to maximize cumulative reward:

- Positive reward for forward displacement along the x-axis
- Large penalty (approximately -100) for falling
- Small penalties for applying motor torque (energy cost)

An episode terminates when:

- The robot falls (torso contacts ground), or
- The end of the terrain is reached, or
- (The time limit is exceeded)

2. Theory

2.1. Deep Neural Networks

Deep Neural Networks (DNNs) consist of an input layer, multiple hidden layers, and an output layer. Each layer contains nodes (neurons) connected to the next layer by weights θ and biases, which are updated during training to minimize a loss function.

2.1.1. Universal Function Approximators

A feedforward network with at least one hidden layer and a non-linear activation function can approximate any continuous function to arbitrary precision, given sufficient capacity (width).

2.1.2. Hierarchical Feature Representation

As data passes through a deep network, each successive layer extracts increasingly abstract features. This essentially can be considered as automatic feature engineering. (=extracting critical components required for decision-making) It facilitates compression and therefore generalization.

2.1.3. Scaling and Performance

Increasing a network's width or its depth, increases its capacity to represent complex functions. While deeper networks are generally more parameter-efficient than wide, shallow ones, they are also harder to train.

2.2. Reinforcement Learning

Reinforcement Learning is a type of Machine Learning where the agent learns by trial and error to optimize its behaviour for a return heuristic, as opposed to being trained on labeled examples.

The return heuristic is calculated over episodes, which are sequences of interactions between agent and given environment, subject to evaluation. In RL episodes is modelled by Markov Decision Processes. We call specific realizations of episodes trajectories.

2.2.1. Markov Decision Process - MDP

The MDP is a sequence of states s_i , actions a_i and rewards r_i

$$s_1, a_1, r_1, s_2, a_2, r_2, s_3, a_3, r_3, \dots$$

where each action a_i is taken (in some way) based on the state s_i , results in a state transition $s_i \rightarrow s_{i+1}$ (controlled by the environment) and the reward r_i (comes from environment).

In particular the MDP must follow Markov's Property

Markov's Property. Observations are markovian if each subsequent state solely depends on the previous state and an action or in other words, the previous state and taken action are sufficient to determine the subsequent state.

$$P(s_{t+1}|s_t, a_t) = P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0)$$

2.2.2. Return G_t

The return at (s_t, a_t) is a function of the subsequent rewards:

$$G_t = G_t(r_t, r_{t+1}, \dots)$$

For example it can be defined as discounted sum of future rewards $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$, with $\gamma \in [0, 1]$.

2.2.3. Policy $\pi(a_i|s_i)$

The model's objective is to learn of a policy $\pi(a_i|s_i)$, which for each (s_t, a_t) picks an action such the respective expected return is maximized. (Environment may be stochastic)

2.2.4. World Models

The environment's internal mechanics are defined by the transition dynamics $P(s_{t+1}|s_t, a_t)$ and the reward function $R(s_t, a_t)$. Together, these are known as the World Model.

- **Model-Free RL:** No access to transition and reward functions. Agent learns strictly from experience.
- **Model-Based RL:** Agent learns or is given a World Model. Thus it is able to simulate future trajectories without interacting with the environment (=planning).

2.2.5. Value Functions

To solve an MDP, the agent must estimate the "goodness" of being in a particular state or taking a particular action. This can be formalized with value functions, which estimate the expected return.

State-Value Function $V^\pi(s)$.

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s] \quad (1)$$

Action-Value Function (Q-Function) $Q^\pi(s, a)$.

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t | s_t = s, a_t = a] \quad (2)$$

Optimal Value Functions V^ and Q^* .*

$$Q^*(s, a) = \max_{\pi} Q^\pi(s, a) \quad (3)$$

and V^* analogously.

For Value-Based methods like DQN, the Q-function is the primary focus as it allows the agent to rank actions without needing a model of the environment's transitions. Optimal Value Functions resemble the "goal" of RL- They represent the maximum expected return achievable by any policy: If an agent learns Q^* , it can derive the optimal policy by simply selecting the action that maximizes the Q-value in any given state: $\pi^*(s) = \arg \max_a Q^*(s, a)$.

2.2.6. Bellman Equation

Value functions can be decomposed into a recursive (=Bellman Equation). It expresses the relationship between the value of a state (or state-action pair) and the value of its successor states. For Q^* , the Bellman optimality equation is given by:

$$Q^*(s, a) = \mathbb{E}[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a] \quad (4)$$

This equation is the foundation of the DQN algorithm, as it provides a way to iteratively update the estimate of Q^* based on immediate rewards and the predicted future value.

2.2.7. Learning Methods

There are two primary ways an agent can learn to estimate the value functions from experience: Monte Carlo methods and Temporal Difference learning.

Monte Carlo (MC) Methods. Monte Carlo methods learn from complete episodes. The agent interacts with the environment until a terminal state is reached, calculates the total return G_t , and then updates the value of the states visited. While MC is unbiased, it requires the task to be episodic and can suffer from high variance.

Temporal Difference (TD) Learning. TD learning combines ideas from Monte Carlo and Dynamic Programming. Unlike MC, TD methods can learn from incomplete sequences by "bootstrapping" a guess based on another guess.

In TD(0), the simplest form of TD learning, the update target for a state s is the immediate reward plus the discounted value of the next state:

$$Target = r_{t+1} + \gamma V(s_{t+1}) \quad (5)$$

DQN utilizes TD learning, as it updates its Q-network at every time step using the transition (s, a, r, s') , making it significantly more sample-efficient than Monte Carlo approaches.

2.3. Deep Q-Networks (DQN)

Deep Q-Networks represent the fusion of the above. On the comparative to non-neural (tabular) RL, DQN win, because they are able to compress information (through generalization), whereas tabular RL would have to store all transitions in a LUT. This equivalently means that DQNs are suitable for continuous state spaces, since they can estimate $Q(s, a; \theta) \approx Q^*(s, a)$, allowing to act correctly on previously unseen information. Tabular RL would simply fail due to lack of lookup values.

One major pitfall of DQN (but also tabular RL), is that despite accepting continuous state spaces, their outputs are discrete. In applications like robotics this can lead to jitter and lack of fine grained porcision.

2.3.1. DQN Objective and Training

DQN is a Value-Based, Model-Free algorithm that utilizes Temporal Difference learning. The network is trained by minimizing a loss function derived from the Bellman optimality equation. At each iteration i , the Mean Squared Error (MSE) of the TD error is minimized:

$$L_i(\theta_i) = \mathbb{E} \left[\left(\underbrace{r + \gamma \max_{a'} Q(s', a'; \theta_i^-)}_{\text{TD Target}} - Q(s, a; \theta_i) \right)^2 \right] \quad (6)$$

Where θ_i^- represents the parameters of a "Target Network," a periodic snapshot of the weights used to stabilize training.

2.3.2. Action Selection and Exploration

Since the agent in DQN learns from its own experiences, it faces the *exploration-exploitation* dilemma. Occasionally it is reasonable, to take sub-optimal actions to discover better long-term strategies, in order to learn action-value function.

Greedy Policy. A strictly greedy policy always selects the action with the highest estimated value: $a = \arg \max_{a'} Q(s, a')$. This maximizes immediate reward, but prevents the agent from discovering better actions (lack of exploration).

Epsilon-Greedy (ϵ -greedy). In this standard approach, the agent selects a random action with probability ϵ and the greedy action with probability $1 - \epsilon$. Often, ϵ is annealed (decreased) over time, moving from high exploration to high exploitation as the agent's confidence in its Q-estimates grows.

Boltzmann (Softmax) Exploration. Instead of choosing purely at random, Boltzmann exploration assigns a probability to each action based on its Q-value using a temperature parameter τ :

$$P(a|s) = \frac{e^{Q(s,a)/\tau}}{\sum_{a'} e^{Q(s,a')/\tau}} \quad (7)$$

This ensures that actions with higher estimated values are explored more frequently than those with very low values.

2.3.3. Experience Replay Buffer

A core challenge in using DNNs for RL is that sequential samples in a trajectory are highly correlated, violating the *i.i.d.* assumption required for stable gradient descent. Furthermore, the agent might "forget" rare but important experiences once it moves to a different part of the state space.

To address this, DQN utilizes an **Experience Replay Buffer** $\mathcal{D}$.

1. During interaction, agent stores transitions $e_t = (s_t, a_t, r_{t+1}, s_{t+1})$ in $\mathcal{D}$.
2. During training, instead of picking most recent transition, agent samples a "mini-batch" of experiences uniformly at random from the buffer.

This mechanism breaks the temporal correlation of data. It allows the network to learn from each experience multiple times, significantly improving sample efficiency and numerical stability.

2.3.4. Target Networks and the Moving Target Problem

A significant reason for instability in Deep Q-Learning arises from the same network parameters θ being used to calculate both the current estimate $Q(s, a; \theta)$ and the TD target. Because the target depends on the network's own predictions:

$$\text{Target} = r + \gamma \max_{a'} Q(s', a'; \theta) \quad (8)$$

any update to θ that reduces the error for the current state also changes the target for the next state. This creates a feedback loop where the "ground truth" the network is trying to learn is constantly shifting, leading to oscillations or divergence.

To solve this *moving target problem*, DQN introduces a second neural network called the **Target Network**, with its own parameters θ^- .

Mechanism. The target network is an identical copy of the online (policy) network but its weights are "frozen." Its exclusive use is to calculate the TD target:

$$y^{DQN} = r + \gamma \max_{a'} Q(s', a'; \theta^-) \quad (9)$$

The online network θ is updated at every training step, while the target network θ^- remains stationary. Periodically (every C steps), the parameters of the online network are copied to the target network: $\theta^- \leftarrow \theta$.

Advantage. By keeping the target fixed for a substantial number of iterations, the reinforcement learning problem is temporarily reduced to a supervised learning task with a stable objective. This means that weight updates are decoupled from target calculation, which reduces oscillations and instability.

3. Implementation

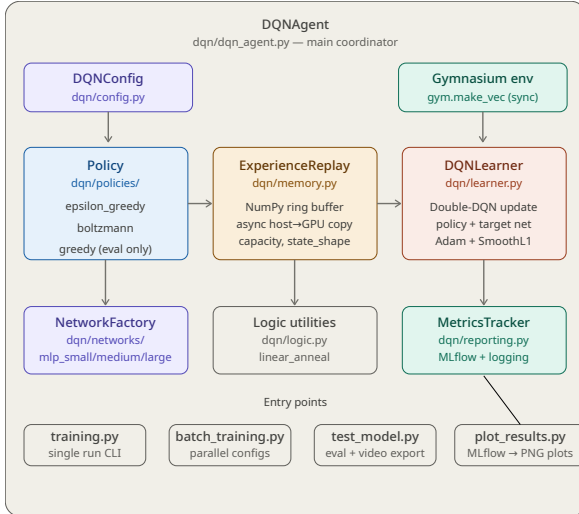


Figure 1: Architecture Diagram

The agent is implemented as a modular Python package (`src/dqn/`) whose components map cleanly onto the standard DQN algorithm. Figure 1 gives an overview.

3.1. Configuration

All hyperparameters are declared in `DQNConfig` (`dqn/config.py`), a typed dataclass whose field names match JSON config file keys one-to-one. At runtime, `DQNConfig.load` resolves a base JSON file (per environment) and applies any CLI overrides, ensuring a single source of truth for every tunable parameter.

3.2. Policy

Action selection is handled by the `BasePolicy` abstract class (`dqn/policies/`), from which three concrete strategies inherit:

- **Epsilon-greedy** — with probability ϵ selects a uniformly random action; otherwise acts greedily. ϵ is annealed linearly from `explore_start` to `explore_end` over training.
- **Boltzmann** — samples from $\text{softmax}(Q(s, \cdot)/T)$, where temperature T is annealed analogously.

- **Greedy** — pure exploitation; used only during evaluation.

Each policy owns its Q-network and performs a non-blocking host-to-device copy to minimize PCIe transfer overhead during the collection phase.

3.3. Replay Memory

`ExperienceReplay` (`dqn/memory.py`) implements a standard ring buffer backed by pre-allocated NumPy arrays. Transitions (s, a, r, s', d) are stored as contiguous blocks; when the buffer wraps around its capacity N older experiences are overwritten. Sampling uses asynchronous `Tensor.copy_` calls to overlap GPU transfer with CPU work.

3.4. Learner

`DQNLearner` (`dqn/learner.py`) maintains a *policy network* Q_θ and a *target network* Q_{θ^-} (a periodically updated copy). It implements Double DQN [hasselt2016deep]: the policy network selects the greedy action in the next state, while the target network evaluates it:

$$y_t = r_t + \gamma Q_{\theta^-} \left(s_{t+1}, \arg \max_{a'} Q_\theta(s_{t+1}, a') \right) (1 - d_t) \quad (10)$$

Parameters are updated via Adam with the Huber (Smooth- L_1) loss between $Q_\theta(s_t, a_t)$ and y_t . The target network is synchronized every `target_sync_interval` epochs; hard copies ($\tau = 1$) are used by default, with optional Polyak averaging for $\tau < 1$.

3.5. Neural Networks

The Q-function is approximated by a feedforward MLP (`dqn/networks/feedforward.py`) with ReLU activations. Three sizes are registered in `NetworkFactory`:

Key	Hidden layers
mlp_small	128 → 128
mlp_medium	256 → 128 → 64
mlp_large	512 → 256 → 128 → 64

Input is the 24-dimensional observation vector; output is a Q-value per discrete action.

3.6. Training Loop

`DQNAgent.train` (`dqn/dqn_agent.py`) orchestrates the following phases each epoch:

1. **Warmup** — random actions fill the replay buffer for `warmup_steps` steps before any gradient updates occur.
2. **Collection** — the current policy interacts with `num_envs` parallel environments for `steps_per_epoch` steps, storing transitions in the buffer.

3. **Optimization** — `updates_per_epoch` mini-batches of size `batch_size` are sampled and used for gradient steps.
4. **Target sync** — the target network is updated every `target_sync_interval` epochs.

Exploration is annealed as a function of `current_step` / `total_steps` via the shared `linear_anneal` utility (`dqn/logic.py`). Metrics (loss, mean episode reward, mean episode length) are logged per epoch to MLflow via `MetricsTracker`.

3.7. Action Space Discretisation

Since DQN requires a discrete action space and BipedalWalker-v3 natively exposes a continuous $[-1, 1]^4$ action space, a wrapper (`env/bipedal_wrapper.py`) maps a finite set of motor-speed combinations to integer indices. The discretisation scheme is described in Section ??.

4. Training

5. Results